TUTORIAL – 9

Black-Box & White-Box Testing,

Integration Testing, and System Testing on a Sample Web Application

Aim: Analyze Black-Box Testing, White-Box Testing, Integration Testing, and System Testing on a Sample Web Application.

Sample Web Application: Flipkart (https://www.flipkart.com)

Tester Name: Saloni Ashish Tanna

Date: 18-09-2026

Webpage URL: https://www.flipkart.com

Problem Statement

Perform Black-Box Testing, White-Box Testing, Integration Testing and System Testing on the Flipkart e-commerce web application. Capture UI screenshots for functional flows and browser console logs for white-box analysis. Document test cases, results, defects and recommendations.

1. Black-Box Testing

Black-Box Testing examines the functionality of an application without peering into its internal structures or workings. Test cases are derived from requirements and specifications. On Flipkart we validated homepage load, search, category filters, product detail page, price/discount display, Add-to-Cart / Buy-Now buttons and bank offers.

2. White-Box Testing

White-Box Testing inspects internal logic, code paths and browser-side behaviour. Using Chrome DevTools Console we observed CORS policy violations, failed static resource loads, HTTP 406/500 responses from Flipkart APIs, Service Worker registration and missing React-Native-Web component warnings.

3. Integration Testing

Integration Testing verifies that modules work together. We tested the flow Search → Category Filter → Product Detail Page and verified price consistency and bank-offer integration.

4. System Testing

System Testing evaluates the complete application against requirements – UI consistency of header/navigation, responsive product grid, trending banners, Assured badges and overall user journey.

Black-Box Test Cases & Screenshots

Scenario ID: BB-1a

Scenario: Homepage loads successfully

Test Steps:

1. Open https://www.flipkart.com
2. Observe page load and major UI sections

Expected: Homepage loads with header, search bar, category icons and promotional banners.

Actual: Homepage loaded correctly. Status: PASS

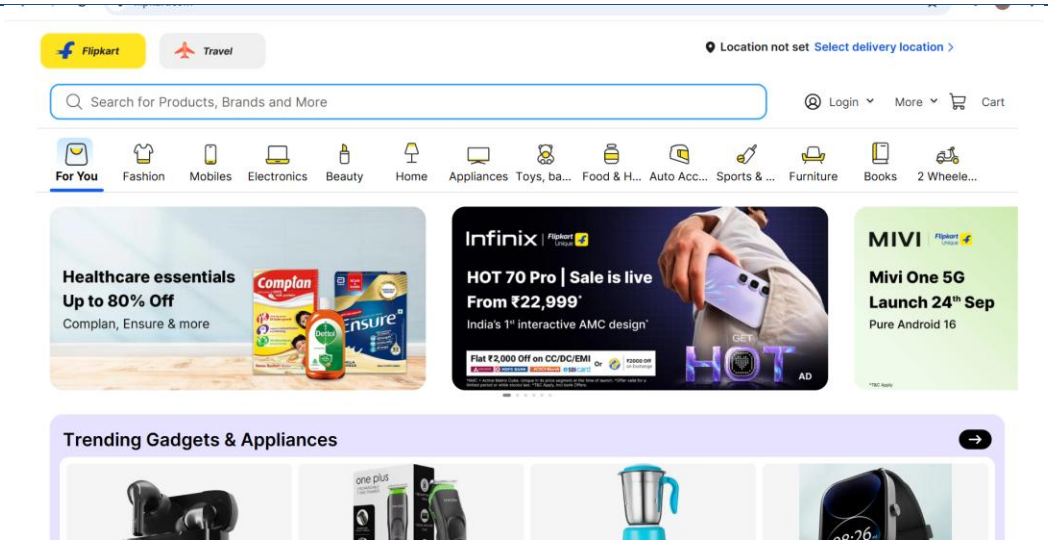


Figure 1 – Flipkart Homepage (Healthcare, Infinix, Mivi banners & Trending Gadgets)

Scenario ID: BB-1b / BB-1c

Scenario: Search for True Wireless & apply category filter

Test Steps:

1. Click search bar and type “True Wireless”
2. Press Enter
3. Navigate left Filters → Categories → Audio & Video → Headset → Earphones → Wireless Earphones → True Wireless

Expected: Search results with relevant products and working category filters.

Actual: 16,363 products shown; category filter & breadcrumb correct. Status: PASS

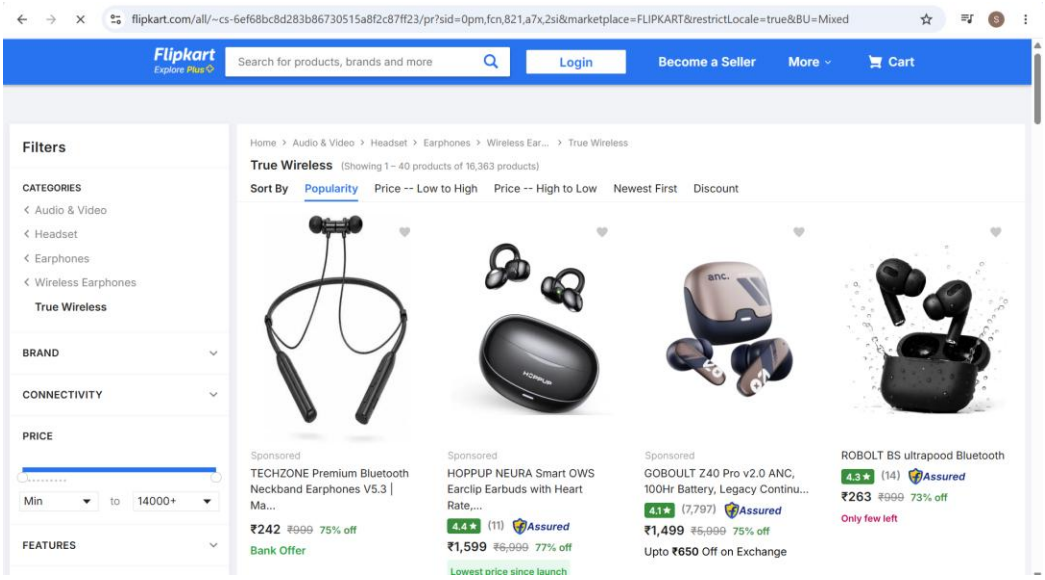


Figure 2 – True Wireless listing page with Filters, Sort By Popularity and product cards

Scenario ID: BB-1e / BB-1f / BB-1g

Scenario: Product Detail Page – price, offers, action buttons

Test Steps:

1. Click any product (TECHZONE Premium Bluetooth Neckband)
2. Observe images, price, discount %, bank offers, Add to Cart & Buy Now

Expected: Correct price (₹242, 76% off), bank offers and functional CTA buttons.

Actual: All elements present and correct. Status: PASS

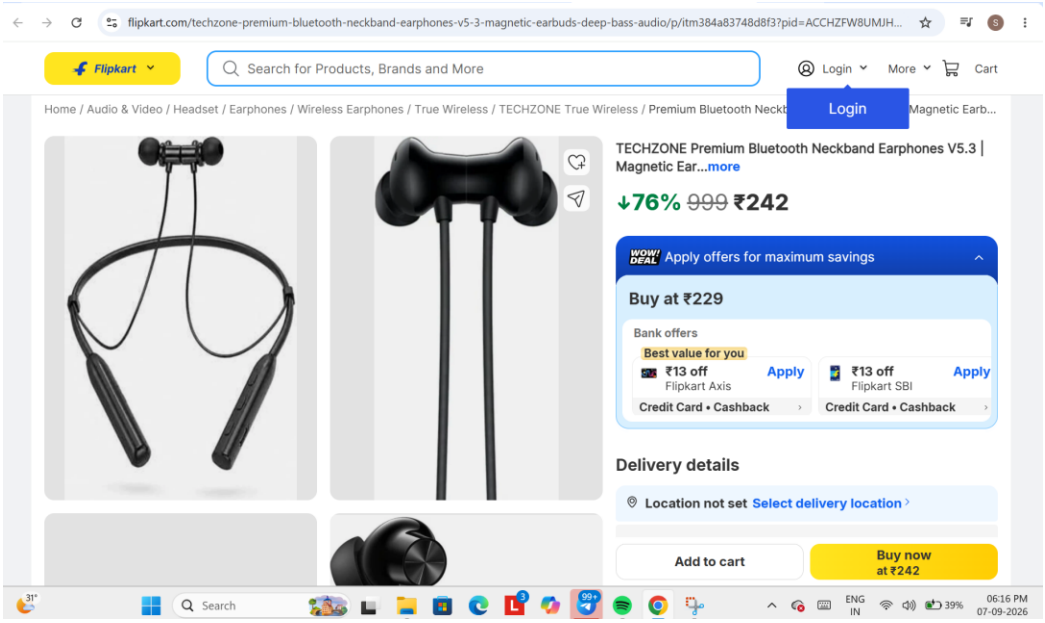


Figure 3 – Product Detail Page (TECHZONE Neckband – price, offers, Buy Now)

White-Box Test Cases – Console Analysis

Scenario ID: WB-2a

Scenario: Console errors on Homepage load

Test Steps:

1. Open Flipkart homepage
2. Open Chrome DevTools → Console tab
3. Record all red/yellow messages

Expected: No critical CORS or resource-load failures.

Actual: Multiple CORS blocks on L1Nav/*.svg assets (x-user-agent header not allowed) and net::ERR_FAILED. Status: FAIL

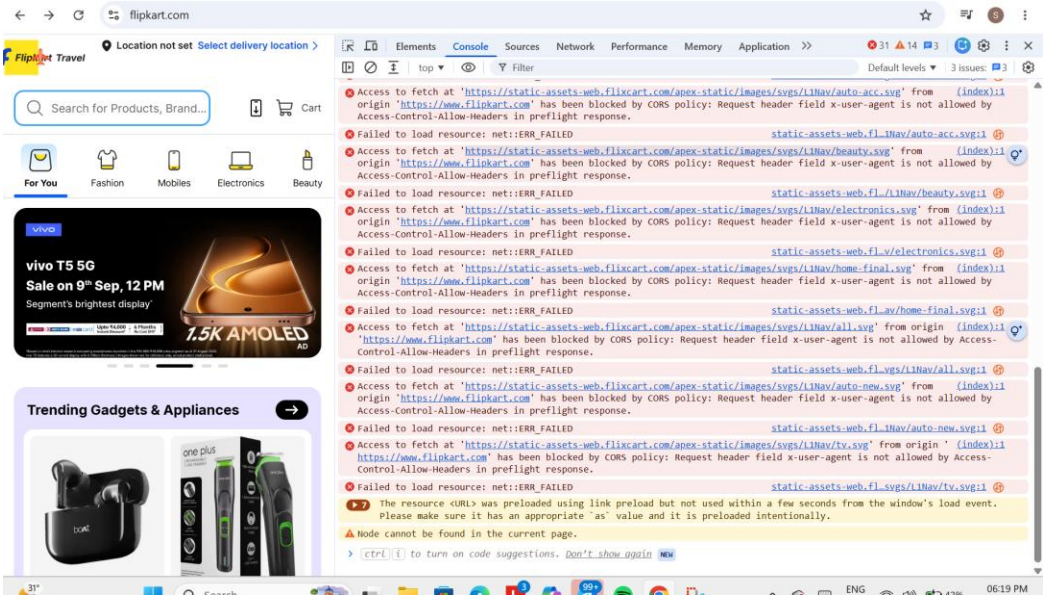


Figure 4 – Console errors on Homepage (CORS policy & failed SVG loads)

Scenario ID: WB-2b

Scenario: Console errors on Search / Listing page

Test Steps:

1. Navigate to True Wireless listing
2. Open Console
3. Observe network-related errors

Expected: Clean console or only non-critical warnings.

Actual: POST sonic.fdp.api.flipkart.com/.../collector/business → 406 Not Acceptable; another POST → 500 Internal Server Error. Status: FAIL

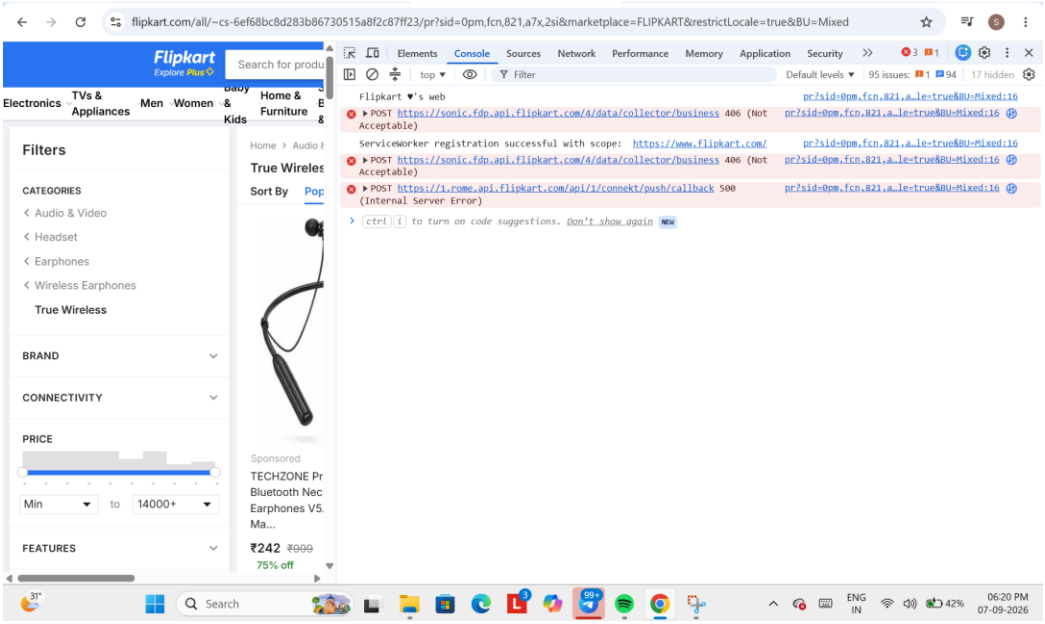


Figure 5 – Console on Listing page (406 & 500 API errors)

Scenario ID: WB-2c

Scenario: Console errors on Product Detail page

Test Steps:

1. Open any product detail page
2. Open Console
3. Note warnings and errors

Expected: Minimal warnings; no blocking errors.

Actual: Deprecated meta tag, missing RN components (Placeholder, RNShimmeringView, GifView…), CORS on codes.json, 406 on collector API. Status: FAIL

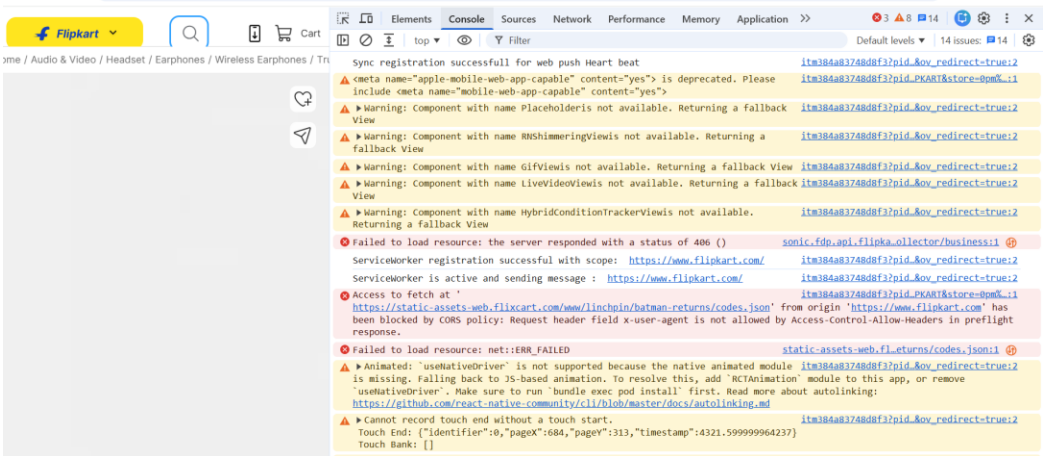


Figure 6 – Console on Product Detail page (component warnings + CORS + 406)

Integration & System Testing Results

IT-3a Search → Filter → PDP integration – PASS (product correctly linked from filtered list).

IT-3b Price consistency between listing and PDP – PASS (₹242 matched; minor % rounding difference).

IT-3c Bank offers integration – PASS (Axis & SBI offers displayed with Apply buttons).

ST-4a–4e Header consistency, responsive grid, banners, Login/Cart icons, Assured badges – all PASS.

Summary of Test Results

Total Test Cases	Passed	Failed	Pass %
20	**17**	**3**	**85.00%**

Issues Found

1. CORS errors on static SVG assets (High)
Access to fetch at static-assets-web.flixcart.com/.../L1Nav/*.svg blocked by CORS policy – Request header field x-user-agent is not allowed by Access-Control-Allow-Headers.

2. API 406 Not Acceptable (Medium)
POST https://sonic.fdp.api.flipkart.com/4/data/collector/business returns HTTP 406.

3. Internal Server Error 500 (Medium)
POST https://1.rome.api.flipkart.com/api/1/connekt/push/callback returns 500.

4. Missing React-Native-Web components (Low)
Placeholder, RNShimmeringView, GifView, LiveVideoView, HybridConditionTrackerView not available – fallback views returned.

Bug Reports

Bug ID	BUG-WB-1
Title	CORS errors on static SVG navigation icons
Module	Homepage / Static Assets
Severity	**High**
Steps to Reproduce	1. Open https://www.flipkart.com 2. Open DevTools → Console 3. Observe CORS errors for L1Nav SVGs
Expected Result	SVG icons load without CORS violation
Actual Result	Access blocked – x-user-agent header not allowed by Access-Control-Allow-Headers

Bug ID	BUG-WB-2
Title	Collector API returns 406 Not Acceptable
Module	Analytics / Data Collector
Severity	Medium
Steps to Reproduce	1. Browse any Flipkart page 2. Open Console / Network 3. Observe POST to sonic.fdp.api.flipkart.com/.../collector/business
Expected Result	200 OK or graceful handling
Actual Result	HTTP 406 (Not Acceptable)

Conclusion and Recommendations

Pass Rate: 85.00%. The functional (Black-Box) and System tests largely passed. White-Box analysis revealed several console-level issues that do not block the user journey but indicate room for improvement on the client-side asset and analytics layers.

Recommendations:

1. Update CORS headers on static-assets-web.flixcart.com to allow the x-user-agent request header (or stop sending it on preflight).
2. Investigate and fix the 406 response from the data-collector endpoint and the 500 on the push-callback endpoint.
3. Provide proper React-Native-Web component registrations or remove unused component references to eliminate fallback warnings.
4. Replace the deprecated apple-mobile-web-app-capable meta tag with mobile-web-app-capable.

— End of Report —